Connecting the Dots

Speaker Notes — RISE Deep Dive

RISE

The Prompt That Started Everything

Presenter slide 0 2 min **Script:**

- Open with the literal origin story. This was not a clever retrospective framing — it was the *actual prompt* typed into ChatGPT at the start of this project.
- Read the prompt aloud verbatim: "Produce mathematics at the level of Vladimir Voevodsky, Fields Medal-winning, foundation-shaking work but directed toward something the legendary Nikolaj Bjørner (co-creator of Z3) could actually use?"
- **Voevodsky** context (one sentence): Fields Medal 2002, inventor of Univalent Foundations / homotopy type theory — the kind of mathematics that rewrites the foundations of all of mathematics.
- **Bjørner** context (one sentence): principal architect of Z3, the world's most widely used SMT solver, a tool that *actually runs* in production at Microsoft, Amazon, and every major verification team.
- The ask was intentionally absurd: foundations-of-math caliber rigor, practical solver-level utility. We expected slop.
- Pause here. Say: "The AI generated dozens of ideas. Most were slop. One thread survived."

Five “Random” Dots

Presenter slide 1 3 min **Script:**

- Open with the intellectual punchline: this project connects 126 years of seemingly unrelated math. Hilbert (1900), Stengle (1974), Parrilo (2000), Prajna (2004), and then 20 verification techniques we “stole.”
- Frame the talk as an origin story: how did a Python bug finder end up rooted in Hilbert’s 17th Problem?
- Key phrase: “Each of these dots was a different field. Pure math, real algebraic geometry, control theory, formal verification. Nobody planned this thread.”
- Set the audience expectation: we’ll walk through each dot, then show how the barrier obligations turned a paper zoo into a coherent portfolio, and then show contracts (PyTorch) composing with the framework.
- If challenged on novelty: the novelty is not in any single paper — it’s in the **integration discipline** that bent 20 methods into three obligations over Python bytecode.

Dot 1: Hilbert's 17th Problem (1900)

Presenter slide 2 3 min **Script:**

- Start historically. 1900, International Congress of Mathematicians. Hilbert poses 23 problems. Problem 17: is every non-negative polynomial a sum of squares of rational functions?
- Answered YES by Artin in 1927. The answer is constructive: if $p(x) \geq 0$ for all x, then p can be written as $\sum (q_i/r_i)^2$.
- This is **pure** mathematics. No one is thinking about programs, robots, or software verification.
- But the implication is enormous: **non-negativity has a finite algebraic certificate**. If something is always positive, you can *prove* it with algebra.
- This is the seed of everything we do. When we later want to prove "this program variable is never zero at this division," we are implicitly asking Hilbert's question about a polynomial encoding of the program state.
- Anecdote: Hilbert thought this would be a quick problem. It took 27 years to prove and 100+ years to become computationally useful.

Dot 2: Stengle's Positivstellensatz (1974)

Presenter slide 3 3 min **Script:**

- 74 years after Hilbert, Stengle generalizes to **semialgebraic sets**: regions defined by polynomial inequalities ($\geq$, $=$, $\leq$).
- The Positivstellensatz says: if a system of polynomial inequalities has no solution, there is a finite algebraic identity (a "certificate") proving infeasibility.
- In our language: if the set $\{x : g_1(x) \geq 0, \ldots, g_m(x) \geq 0\}$ is empty, there exist polynomials s_i, t_j such that a specific algebraic identity shows the emptiness.
- This is the **existence guarantee** for barrier certificates. If safe states and unsafe states don't overlap, a polynomial fence B separating them *must exist* (under mild conditions).
- Still no programs, still no computers. Stengle is doing real algebraic geometry.
- Key phrase for RISE: "Stengle tells us that if the bug is genuinely unreachable, we are *guaranteed* a proof exists. The question becomes: can we *find* it?"
- How we bent it: we map Python program state variables to the polynomial ring. Guard conditions like `if x > 0` and error conditions like `den == 0` become the semialgebraic sets S_0 and U. Stengle's theorem applies over these mapped sets.

Dot 3: Parrilo's SOS Programming (2000)

Presenter slide 4 3 min **Script:**

- This is the jump into **control theory**. Pablo Parrilo's PhD thesis at Caltech.
- Stengle says a certificate exists. But how do you *find* it? That's an infinite-dimensional search.
- Parrilo's insight: restrict to **sum-of-squares** (SOS) polynomials. A polynomial $p(x) = \sum q_i(x)^2$ is automatically non-negative. Checking if p has an SOS decomposition is equivalent to checking if a **Gram matrix** is positive semidefinite — which is a semidefinite program (SDP).
- SDPs have polynomial-time interior-point algorithms. Suddenly, the existential question ("does a certificate exist?") becomes an optimization problem ("solve this SDP for the coefficients").
- Parrilo's domain was Lyapunov stability for ODEs: prove that a dynamical system (robot, circuit, aircraft) stays in a safe region. The SOS machinery computes Lyapunov functions.
- How we bent it: Parrilo's state space is continuous ODE states ($\dot{x} = f(x)$). We substitute with Python's symbolic machine state $\sigma = (pc, \rho, h, \tau, \kappa, e, \Phi)$. The SDP solver doesn't care what the variables *mean* — it just solves for coefficients.

Dot 4: Prajna's Barrier Certificates (HSCC 2004)

Presenter slide 5 3 min **Script:**

- This is the jump from **control theory to verification**. Prajna and Jadbabaie, HSCC 2004.
- Lyapunov functions prove *stability* (system converges to an equilibrium). Barrier functions prove *safety* (system never enters a bad region). Different question, same SOS machinery.
- Prajna wrote down three explicit obligations that a barrier function B must satisfy:
  1. **Init:** $\forall s \in S_0 : B(s) \geq \epsilon$ (barrier is positive at start)
  2. **Consecution:** $B(s) \geq 0 \Rightarrow B(s') \geq 0$ (barrier stays non-negative along transitions)
  3. **Separation:** $\forall s \in U : B(s) \leq -\epsilon$ (barrier is negative on unsafe states)
- If all three hold, no execution path can cross from S_0 to U. The unsafe states are provably unreachable.
- This is *the* paradigm we use. Every other paper in the portfolio is a specialized solver for one or more of these obligations.
- How we bent it: Prajna uses Lie-derivative consecution ($\dot{B} \geq 0$ along the ODE flow). We replace this with symbolic one-step consecution: $B(\sigma') \geq 0$ under the discrete transition relation T derived from Python bytecode semantics.
- Key insight for RISE: the three obligations are not just a proof structure — they are an

Dot 5: The 20 Techniques We Stole

Presenter slide 6 2 min **Script:**

- With the barrier interface in hand, every verification technique becomes a specialized subsolver.
- SOS/SDP solves init and separation. IC3/PDR solves consecution. CEGAR refines the template when it fails. Houdini proposes coefficient candidates. Concolic confirms with real executions.
- The framework isn't "use everything and hope." It's structured decomposition: each method either **validates** an obligation, **refutes** a candidate, or **narrows the template space**.
- Failure is informative: if SOS can't certify separation, that tells you the unsafe boundary is outside polynomial reach for the current variable basis. That failure is a routing signal, not a dead end.
- Transition to the next slides: "Now let me show you the connection from Hilbert to Python in one picture, then we'll dig into why this matters practically."

Connecting the Dots

Presenter slide 7 2 min **Script:**

- Walk through the visual: 1900 (pure math) → 1974 (certificates) → 2000 (computable via SOS) → 2004 (verification API) → 2024–2026 (Python portfolio).
- Each transition looked unreasonable at the time. Hilbert → Stengle took 74 years. Stengle → Parrilo required a new field (SDP optimization). Parrilo → Prajna required reinterpreting stability as safety. Prajna → us required replacing ODE dynamics with Python bytecode transitions.
- The punchline: "126 years of seemingly unrelated work. Each dot was a different field. Together they form a complete Python bug-finding pipeline."
- RISE-specific: stress that this is not revisionist history. We did not plan this thread. The AI generated candidate ideas, and the surviving ones turned out to trace this lineage. The connection was discovered, not designed.

Why a New Theoretical Basis?

Presenter slide 8 3 min **Script:**

- Anticipate the obvious question: "Existing verification methods work. Abstract interpretation, model checking, SMT solving — why not just use them?"
- Answer: each works well **in isolation**. The problem is **composition**. Abstract interpretation has no standard API for combining with IC3. SMT solving produces models but not structured proof artifacts. Model checking hits state explosion on numeric programs.
- What barrier theory adds is **not one more method**. It's the **interface** that lets all methods cooperate.
- Three obligations = three questions. Any method that answers one of those questions is composable with any method that answers another. This is why we can have SOS handling separation, IC3 handling consecution, and CEGAR handling refinement, all working on the *same* certificate B.
- Failure routing: when SOS fails on separation, that's informative data for CEGAR. When IC3 fails on consecution, that's a signal to try CHC. Each failure maps to a *specific* obligation that tells you what to try next.

The Unifying Interface

Presenter slide 9 2 min **Script:**

- Walk through the diagram: three obligation boxes at the bottom (Init, Consecution, Separation), with methods feeding into each one from above.
- SOS/SDP and DSOS handle Init and Separation (polynomial positivity/negativity checks).
- IC3/PDR, CHC/Spacer, and k-induction handle Consecution ("does B stay non-negative along transitions?").
- CEGAR, Interpolants, Houdini/ICE handle refinement when obligations fail ("how do we improve the template?").
- Stress: composition is **structural**, not ad hoc. Every method serves the same three-obligation contract. This is what turns a paper zoo into a coherent verification pipeline.
- RISE point: if someone asks "but what about method X?" — the answer is always "which obligation does it discharge?" If it discharges one, it slots in. If it doesn't, it's not useful here.

Why Barriers Beat the Status Quo

Presenter slide 10 2 min **Script:**

- Three-column comparison: Linters (fast, no proofs, 80–95% FP rate), Testing/Fuzzing (finds bugs but can never prove absence, 2^{64} inputs), Barrier certs (mathematical proof, path-sensitive, handles loops, eliminates silently).
- On real repos: 99% of candidate bugs proven unreachable. Only $\sim$1% survive for triage.
- The operational value is not "we have a cool theorem." It's "developers see 3 findings instead of 300, and the 3 are real."
- If asked about false negatives: barrier certificates are one-sided. If we prove something unreachable, that's sound. If we *can't* prove it, we keep the candidate conservatively. We never drop something we can't prove safe.

Proof, Not Confidence

Presenter slide 11 2 min **Script:**

- Walk through the comparison table. The key rows:
- "Proves absence": only barrier certificates give you this. LLM triage gives confidence scores; linters give pattern matches. Neither is a proof.
- "Auditable artifact": the barrier certificate is a proof object: polynomial template, coefficients, which obligations were checked, solver outcome for each. You can replay it independently.
- "Handles loops": this is where pattern matchers and LLMs fundamentally fail. A loop may have a complex invariant that protects a dangerous operation. Only inductive reasoning (barrier consecution) can capture this.
- "Scales to CI": with DSOS as first tier, barrier checks take seconds per function. This is CI-compatible. Full SOS escalation is reserved for hard cases.

Concrete Example: DIV_ZERO with Polynomial Barrier

Presenter slide 12 3 min **Script:**

- Walk through the `safe_divide` example step by step.
- State variable: $x = b$ (the denominator). This is the only variable relevant to the division safety.
- Unsafe set: $U = \{x : x = 0\}$. If $x = 0$ at the division, we crash.
- Guard: the `if b != 0` check means the division is only reachable when $x \neq 0$.
- Template choice: $B(x) = x - 1/2$. This is a degree-1 polynomial.
- Check obligations:
  1. Init: after the guard, for integers, $|x| \geq 1$, so $B(x) \geq 1/2 > 0$. ✓
  2. Consecution: x is not modified between the guard and the division, so B is preserved. ✓
  3. Separation: $B(0) = -1/2 < 0$. The barrier is negative at the unsafe point. ✓
- All three hold $\Rightarrow$ DIV_ZERO is provably unreachable. This finding is eliminated as an FP with a proof artifact.
- Emphasize: "This is Hilbert $\rightarrow$ Stengle $\rightarrow$ Parrilo $\rightarrow$ Prajna, instantiated on one Python function. The 126-year thread works."
- If asked about more complex examples: for multi-variable cases, the polynomial has cross-terms and degree > 1. SOS synthesizes the coefficients via SDP. The principle is

The Polynomial Fence, Visually

Presenter slide 13 1 min **Script:**

- Quick visual reinforcement. The barrier $B(x) = x - 1/2$ is a line crossing zero at $x = 1/2$.
- Safe region (where $B \geq 0$): $x \geq 1/2$. This is where guarded code executes.
- Unsafe point ($x = 0$): $B(0) = -1/2 < 0$. Separated by the fence.
- No execution can cross from safe to unsafe without B going through zero — and consecution guarantees B never drops below zero on valid transitions.
- This is the geometric intuition for barrier certificates. In higher dimensions, the fence is a surface; the principle is the same.

The Portfolio: 20 Papers, One Framework

Presenter slide 14 2 min **Script:**

- Overview slide. Six layers, each serving the barrier framework:
- **Layer 1** (Foundations): Stengle, Parrilo, Lasserre, Sparse SOS — mathematical existence and computability.
- **Layer 2** (Core): Prajna hybrid/stochastic barriers, SOS safety checker, SOSTOOLS — the three-obligation API.
- **Layer 3** (Speed): DSOS/SDSOS, sparsity, CBF contracts — making it fast enough for CI.
- **Layer 4** (Consecution): k-induction, IC3/PDR, CHC — handling consecution when polynomial templates fail.
- **Layer 5** (Refinement): CEGAR, SLAM, interpolants, lazy abstraction — improving templates when obligations fail.
- **Layer 6** (Synthesis): Houdini, ICE, SyGuS, DART — proposing coefficients and grounding with real executions.
- Transition: "Let me now walk through each paper and show exactly how we bent it."

#1 Stengle 1974 — Positivstellensatz

Presenter slide 15 1.5 min **Script:**

- Already covered in Dot 2. Quick recap: "If safe $\cap$ unsafe $= \varnothing$, a polynomial fence exists."
- Barrier role: theoretical bedrock. We *know* a proof exists before searching.
- How we bent it: pure real algebraic geometry with no program semantics. We map program state variables to the polynomial ring; interpret guards and error conditions as semialgebraic sets.
- For RISE: this is the completeness guarantee (under Archimedean conditions on the variable domains).

#2 Parrilo 2000 — SOS via SDP

Presenter slide 16 1.5 min **Script:**

- Already covered in Dot 3. "Fence exists $\rightarrow$ solve for coefficients via SDP."
- The SDP doesn't care that our variables are Python locals instead of ODE states.
- Key detail: monomial basis is derived from symbolic state projections at the bug site, not from global algebraic variable sets. This keeps the SDP small.
- How we bent it: Parrilo's domain is Lyapunov for continuous nonlinear control. We substitute with discrete symbolic machine state σ.

#3 Lasserre 2001 — Moment/SOS Hierarchy

Presenter slide 17 1.5 min **Script:**

- Originally a polynomial *optimization* framework using moment-SOS duality.
- We repurpose as a *feasibility-escalation* schedule: degree-2 first, then degree-4 if needed, etc.
- Each level is a portfolio tier, not an optimization step. Convergence guarantee means no degree ceiling stops us permanently.
- How we bent it: optimization $\rightarrow$ feasibility check; escalation is budget-controlled.
- Practical note: in practice, degree-2 or degree-4 suffices for $> 90\%$ of Python bug sites.

#4 Sparse SOS (Waki et al., 2006)

Presenter slide 18 1.5 min **Script:**

- Critical for scalability. A Python function might have 10–20 local variables. Without sparsity, the SDP has $\binom{n+d}{d}^2$ entries — intractable.
- Waki et al. show: decompose along the variable correlation graph. Variables that never co-appear in constraints can be factored into separate sub-SDPs.
- How we bent it: original graph from polynomial support structure; we build from Python data-flow. Two variables co-appear iff they share a branch predicate or arithmetic expression.
- Result: many small SDPs instead of one huge one. Makes barrier synthesis tractable for real functions.

#5 Prajna & Jadbabaie, HSCC 2004

Presenter slide 19 1.5 min **Script:**

- Already covered in Dot 4. The three-obligation API: Init, Consecution, Separation.
- This is the paradigm. Every other paper serves these three conditions.
- How we bent it: Lie-derivative consecution ($\dot{B} \geq 0$ along ODE flow) $\rightarrow$ symbolic one-step $B(\sigma') \geq 0$ under discrete transition relation T.
- RISE insight: the obligations are not just proof structure — they are a **composition API**. Any method that discharges one obligation is composable with any method that discharges another.

#6 Prajna et al., TAC 2007 — Stochastic Barriers

Presenter slide 20 1.5 min **Script:**

- Extends barriers to stochastic systems. Consecution becomes $\mathbb{E}[B(s') \mid s] \geq 0$.
- How we bent it: replaces Itô calculus over stochastic differential equations with Python probabilistic branching. Expectation is computed over path-condition probability mass.
- Practical use: risk-aware scoring for paths with inherent non-determinism (e.g., random initialization, hash-dependent branching).
- This is the only part of the framework that produces probabilistic rather than deterministic verdicts.

#7–8 SOS Safety + SOSTOOLS

Presenter slide 21 1 min **Script:**

- #7 (SOS Safety): fast feasibility pre-check. "Can *any B* separate these regions?" If not, skip expensive synthesis entirely.
- #8 (SOSTOOLS): engineering framework — configurable degree, variable selection, parametric template interface. Production plumbing.
- Both serve to make SOS synthesis practical and configurable for CI throughput.
- These are "unglamorous but essential" papers. They turn the math into software.

#9 DSOS/SDSOS (Ahmadi & Majumdar, 2019)

Presenter slide 22 2 min **Script:**

- Key CI performance paper. "Same fence, cheaper paint."
- DSOS uses LP (linear program) instead of SDP. SDSOS uses SOCP (second-order cone).
- Both are *sound subsets* of SOS: DSOS $\subset$ SDSOS $\subset$ SOS $\subset$ nonneg. polys. They never certify something false. They may miss valid certificates that SOS would find.
- 50–200$\times$ faster in practice. In CI, that's 10 seconds vs. 30 minutes per repo scan.
- Portfolio strategy: DSOS first. If DSOS fails, try SDSOS. If SDSOS fails, escalate to full SOS/SDP.
- How we bent it: originally robotics trajectory optimization. We discard the control objective entirely; use DSOS purely as a certificate-existence check.
- RISE Q&A: "Why not just SOS everywhere?" Answer: DSOS at CI speed lets us run on every push. SOS would require nightly-only analysis.

#10 CBF Survey (Ames et al., 2019)

Presenter slide 23 1.5 min **Script:**

- Control Barrier Functions: continuous consecution as $\dot{B} \geq -\alpha(B)$ (Lie derivative with class-$\mathcal{K}$ bound).
- This motivates how we handle unknown/library calls. If we can't analyze a function's internals, we treat it like a "black-box dynamics" with a contract bounding how much B can decrease.
- How we bent it: smooth continuous dynamics with computable Lie derivatives $\rightarrow$ discrete per-call contract $B(\sigma') \geq B(\sigma) - \alpha(B(\sigma))$ for black-box Python callees.
- This is the theoretical bridge to the contracts section later in the talk.

#11 Correlative Sparsity in Practice

Presenter slide 24 1 min **Script:**

- Visual: dense SDP (all variables coupled) vs. sparse (many small SDPs from variable clusters).
- Variables in different branches rarely interact. This is *especially* true in Python where functions tend to have independent local computations.
- Quick stat: for a 15-variable function, sparse decomposition typically produces 4–6 sub-SDPs of 2–4 variables each, instead of one 15-variable SDP.

#12 k-Induction (Sheeran et al., 2000)

Presenter slide 25 1.5 min **Script:**

- When polynomial templates are low quality, k-induction is a fallback consecution checker.
- Base: does any path of length $\leq k$ violate $B \geq 0$? Step: assuming the last k steps maintained $B \geq 0$, does one more step maintain it?
- How we bent it: original is propositional SAT over hardware bit-vectors. We run SMT over integer/real program variables. Same base/step structure, different logic engine.
- Practical use: effective for bounded loops where the induction depth k matches the loop bound.

#13 IC3 (Bradley, 2011)

Presenter slide 26 1.5 min **Script:**

- Builds the barrier *implicitly* as a conjunction of learned clauses (frame predicates).
- Each "blocked cube" is one constraint preventing unsafe states from being reachable at time step i.
- How we bent it: Boolean SAT over hardware registers $\rightarrow$ LIA predicates over Python state. SAT-based cube generalization $\rightarrow$ Z3 model-based quantifier instantiation.
- Key caveat for RISE: IC3's completeness argument does not transfer from SAT to LIA. We accept IC3 as a *heuristic* consecution solver, not a complete one.
- PDR (Eén et al., 2011) is the industrial-scale version; we serialize its inductive clause sequence into the SARIF proof artifact.

#14 CHC/Spacer (Gurfinkel et al., 2015)

Presenter slide 27 1.5 min **Script:**

- Software safety as Constrained Horn Clauses. CHC solving *is* consecution written interprocedurally.
- Head predicates = post-states; body clauses = transitions + summaries. Tighter function summaries = smaller CHC bodies = cheaper solving.
- How we bent it: SeaHorn targets C/LLVM. We re-encode for Python's transition system. Python dynamic dispatch requires per-callsite CHC heads rather than per-function — a new encoding layer SeaHorn doesn't need.
- This is how barrier certificates work *across function boundaries*: interprocedural summaries are CHC predicates tied to barrier obligations at call boundaries.

#15 CEGAR (Clarke et al., 2000)

Presenter slide 28 1.5 min **Script:**

- Failed obligation = gap in the fence. CEGAR: find the gap, add monomials to the template basis, re-solve.
- How we bent it: original "abstraction" is a boolean program; ours is the polynomial template degree and monomial basis. "Refinement" adds monomials from spurious trace path conditions instead of boolean predicates.
- Termination: finite refinement steps (bounded by monomial basis size and degree cap).
- This is the workhorse refinement loop. Most barriers that fail on the first attempt succeed after 1–2 CEGAR rounds.

#16 SLAM (Ball et al., 2001)

Presenter slide 29 1 min **Script:**

- SLAM = CEGAR at Windows driver scale. Proved this approach works in **real software** (not just toy examples).
- We take the *architectural* lesson: automated fence-gap patching scales. We substitute SLAM's boolean predicate engine with a polynomial monomial selector.
- Key RISE point: if SLAM worked at Windows driver scale with boolean predicates, polynomial templates over Python state should be equally tractable.

#17 Craig Interpolants (McMillan, 2003)

Presenter slide 30 1.5 min **Script:**

- Given an UNSAT proof of a spurious trace, extract a Craig interpolant: a formula implied by the prefix and contradicting the suffix. This is the *tightest possible* new predicate for refinement.
- How we bent it: original extracts propositional/linear interpolants for boolean abstraction. We apply SMT interpolation over non-linear arithmetic and map the formula to new monomial terms in the polynomial template.
- The interpolant is derived directly from the failed obligation's UNSAT core. This means refinement is *optimal*: we add exactly the minimal new monomial needed.

#18 Lazy Abstraction (Henzinger et al., 2004)

Presenter slide 31 1 min **Script:**

- Don't rebuild the entire B when one part fails. Surgically patch only the template branch that broke.
- How we bent it: "subtree" in BLAST is a CFG node set in boolean predicate abstraction. In our portfolio, it's the monomial support region of B around the failing obligation check. We localize the SDP re-solve to that variable sub-cluster.
- Practical impact: refinement cost is proportional to the failure, not the program. A failing check in one branch doesn't require re-solving the barrier for the entire function.

#19 Houdini + ICE

Presenter slide 32 2 min **Script:**

- **Houdini (Flanagan & Leino, 2001):** Start with many candidate coefficient configurations. Prune by counterexample. Fixpoint = widest valid B. How we bent: Houdini proposes boolean annotations for ESC/Java; we generate polynomial coefficient candidates. Pruning oracle = three-obligation SMT check.
- **ICE (Garg et al., 2014):** Learn barrier from three kinds of examples: positive (safe states), negative (unsafe states), implication (transition edges). The program's own behavior shapes the fence. How we bent: ICE targets general first-order invariants; we restrict to bounded-degree polynomials and the learner returns c_α coefficient vectors.
- Both solve the "who proposes c_α?" gap that pure SOS leaves open.
- Key: SOS says *what shape B* must have; Houdini/ICE say *which coefficients*.

#20 SyGuS + DART/Concolic

Presenter slide 33 2 min **Script:**

- **SyGuS (Alur et al., 2013):** Template family = grammar. Three obligation checks = oracle. Formalizes our coefficient search. How we bent: SyGuS is generic (strings, bits); we instantiate with a monomial grammar and the three BSP obligations as multi-property SMT oracle.
- **DART/Concolic (Godefroid et al., 2005):** Concrete + symbolic execution. How we bent: DART's goal is *falsification* (find bugs). We *invert* it to *confirmation*: drive concolic runs toward certificate violations to validate $B \geq 0$ on all explored paths and $B \leq -\epsilon$ at real unsafe states.
- Together: SyGuS = formal framework for the search; concolic = ground truth for the certificate.
- RISE summary of papers 17–20: SOS = what shape; Houdini/ICE/SyGuS = which coefficients; concolic = is the certificate faithful?

Contracts: How PyTorch Meets Barriers

Presenter slide 34 3 min **Script:**

- Transition: "We've shown the theory and the portfolio. Now: what happens when the analyzer hits a function it can't analyze — like `torch.cosine_similarity()`?"
- Walk through the example:
  1. Without contract: `sim` could be anything $\rightarrow$ `score` could be anything $\rightarrow$ 0 might be in range $\rightarrow$ false alarm.
  2. With contract: `sim` $\in [-1, 1] \rightarrow$ `score` $\in [-4, -2] \rightarrow 0 \notin [-4, -2] \rightarrow$ provably safe.
- The contract is a **barrier constraint on library outputs**. It constrains what the library function can produce, and that constraint propagates through interval algebra to all downstream operations.
- Connect back to CBF survey (#10): the contract is the discrete analogue of the class-$\mathcal{K}$ slope bound. It says "this call can decrease B by at most this much."
- Soundness principle: $\mathrm{Sem}_f \subseteq R_f$. The contract must *over-approximate* actual behavior. More conservative is always safe; less conservative is unsound (forbidden).
- Default is "havoc" (allow everything). Refinement only with justification (spec, source analysis, or DSE validation).

Contracts Are Barrier Constraints on Library Outputs

Presenter slide 35 2 min **Script:**

- Walk through the visual pipeline: Contract $\rightarrow$ Interval algebra $\rightarrow$ Barrier check.
- Key formula: consecution with contracts becomes $B(s) \geq 0 \wedge T_{\text{prog}}(s, s') \wedge C_{\text{library}}(s, s') \Rightarrow B(s') \geq 0$.
- The library contract C plugs directly into the consecution obligation. This is not ad hoc — it's the same three-obligation framework with library behavior modeled as transition constraints.
- Better contracts $\rightarrow$ tighter intervals $\rightarrow$ more bugs proven unreachable $\rightarrow$ less noise.
- Contract quality is a first-order determinant of FP/TP balance. Too weak $\rightarrow$ more UNKNOWN survivors. Unsound $\rightarrow$ invalid proofs (forbidden).

PyTorch Contract Examples

Presenter slide 36 2 min **Script:**

- Walk through the table of PyTorch contracts.
- `torch.rand`: output $\in [0, 1)$. This excludes negative values from all downstream computations.
- `torch.zeros`: output exactly $= 0$. Known constant.
- `torch.arange(start, end, step)`: precondition step $\neq 0$ — if step could be zero, this **flags** DIV_ZERO. The contract itself catches the bug.
- `cosine_similarity`: output $\in [-1, 1]$. Tight enough for interval propagation to prove many downstream operations safe.
- `view(*shape)`: shape-preserving with precondition $\prod(\text{shape}) = \text{numel}$. Enables shape safety reasoning.
- 1,500+ lines of PyTorch contracts total. Each is a barrier constraint that composes through interval algebra.
- The contracts use a `DimSource` enum to track where dimensions come from: literal constants, function arguments, argument shapes, or computed expressions. This enables semantic shape tracking.

What Actually Shipped

Presenter slide 37 2 min **Script:**

- `pip install a3-python`. Two stages:
- Static stage (bulk): bytecode analysis, Z3 symbolic execution, barrier certificate proofs. 67 bug types. 99% of candidates eliminated with proof artifacts.
- Agentic stage (residue): multi-turn LLM tool-use. Reads files, searches for guards, checks callers and tests. Only inspects the $\sim$1% that survived static analysis. Zero API keys needed in CI (uses `GITHUB_TOKEN`).
- Design principle: proof artifacts are upstream; LLM judgment is downstream. The LLM never overrides a proven elimination.
- CI integration: `a3 init . --copilot` generates GitHub Actions workflows. Every push triggers scan $\rightarrow$ triage $\rightarrow$ baseline ratchet $\rightarrow$ SARIF upload.

The Meta-Lesson

Presenter slide 38 3 min **Script:**

- Walk through the timeline: 1900, 1974, 2000, 2004, 2024–2026.
- Each transition looked unreasonable. Pure math → algebraic geometry → control theory → verification → Python. Five different fields.
- The AI generated candidate ideas across months. Most were wrong. The survivors traced this lineage — the connection was *discovered*, not designed.
- Five key points:
  1. Hilbert asked if non-negative polynomials are sums of squares (1900).
  2. Stengle proved infeasibility has algebraic certificates (1974).
  3. Parrilo made this computable with SDPs — for control theory (2000).
  4. Prajna wrote three barrier obligations — for verification (2004).
  5. We stole 20 techniques, bent them into those three obligations, and applied them to Python bytecode (2024–2026).
- Closing phrase: "The dots were always there. We just drew the line."
- From Hilbert's 17th Problem to your CI pipeline. 126 years. Questions?

Close + Q&A

Presenter slide 39 open **Expected questions:**

- **State encoding:** what is concrete vs. symbolic? Answer: PC and frame-stack are concrete; locals/stack/heap observers are symbolic Z3 terms.
- **Solver policy:** how are UNKNOWN and timeout treated? Answer: kept conservatively; candidate escalates to next portfolio tier.
- **Contracts:** where do unknown-call assumptions live? Answer: in a typed contract registry (`contracts/*.py`); auditable, versioned, tested.
- **Refinement:** what triggers CEGAR and when do you stop? Answer: obligation failure triggers it; stop on proof, witness, or budget boundary.
- **Artifact chain:** can every elimination be explained? Answer: yes — template, coefficients, obligations, solver outcomes, site metadata, all in SARIF.
- **Completeness:** Python dynamism limits full completeness. Explicit assumptions, observable failure modes.
- **Comparison to Infer/Semgrep:** Infer uses separation logic (heap focus); Semgrep uses pattern matching (no proofs). Neither has a portfolio composition API.